

Elements of Software Engineering

Chapter 1 – Overview

Nguyễn Thị Minh Tuyền · Faculty of Information Technology, HCMUS

Objectives

01

Understand what software engineering is and why it is important

02

Understand that different types of software systems may require different SE techniques

03

Understand ethical and professional issues that are important for software engineers

Topics Covered

1

Software

2

**Software
Engineering**

3

Software Process

4

**Software
Engineering Ethics**



1

Software

What it is, why it matters, and how it's classified

What is Software?

Computer programs and their associated documentation

Software is abstract and intangible – unlike physical engineering, it is not constrained by materials or physical laws. This means there are no natural limits to complexity.

Professional software is intended for use by someone other than its developer, is built by teams, and must be maintained throughout its entire life.

A complete software system includes:

Programs & code

Configuration files

**System
documentation**

**User
documentation**

Support resources

Role of Software



The economies of ALL developed nations are dependent on software



More and more systems are software-controlled



Software engineering is concerned with theories, methods, and tools for professional software development

Software Products

GENERIC

Stand-alone systems sold to any customer who wishes to buy them.

The specification of what the software should do is owned by the software developer and decisions on software change are made by the developer.

CUSTOMIZED

Software commissioned by a specific customer to meet their own needs.

The specification of what the software should do is owned by the customer for the software and they make decisions on software changes that are required.

 The line is blurring: ERP systems (e.g. SAP, Oracle) are generic bases adapted to specific business rules – a hybrid of both categories.

Essential Attributes of Good Software



Maintainability

Software must evolve to meet changing customer needs. For long-lived systems, maintenance costs can far exceed initial development costs.



Dependability & Security

Includes reliability, safety, and security. Software must not cause physical or economic damage if it fails.



Efficiency

Avoid wasteful use of system resources such as memory and processor cycles – not just raw speed.



Acceptability

Must be understandable, usable, and compatible with the systems and workflows of its intended users.

2

Software Engineering

Definitions, history, costs, and disciplines

History – The Software Crisis

The term "Software Crisis" was coined at NATO conferences in 1968, recognizing that software was increasingly unreliable, over budget, and hard to maintain.

The engineering approach would:



Reduce the costs of software development



Lead to more reliable, trustworthy software



Apply disciplined engineering methods – not just programming

 Real cost: Ariane 5 rocket exploded on maiden flight (1996) due to a software type conversion error – \$370M lost.

What is Software Engineering?

Software engineering is an engineering discipline concerned with all aspects of software production – from early specification through to maintenance after deployment.

Engineering discipline

Using appropriate theories and methods bearing in mind organizational and financial constraints.

All aspects of production

Not just technical development – also project management, process improvement, tool & method development.

Professional development

Intended for use by others, built by teams, and maintained throughout its entire life – not a personal programming project.

Software Engineering – Layered Technology

Tools

CASE tools, IDEs, version control, testing tools

Methods

Agile, Waterfall, RUP – how to build software

Process Model

Framework that must be established for effective delivery of software engineering technology

Quality Focus

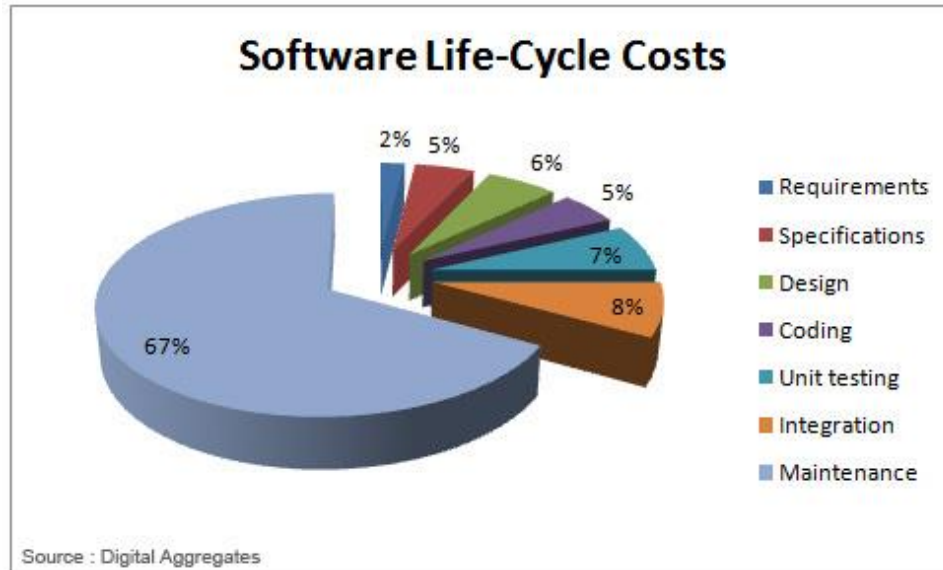
The bedrock – a commitment to quality drives everything above

Software costs

- Software costs often dominate computer system costs. The costs of software on a PC are often greater than the hardware cost.
- Software costs more to maintain than it does to develop. For systems with a long life, maintenance costs may be several times development costs.
- Software engineering is concerned with cost-effective software development.

Costs of software engineering

- Roughly 60% of software costs are development costs, 40% are testing costs.
- For custom software, evolution costs often exceed development costs.



SE vs Computer Science vs System Engineering

Computer Science

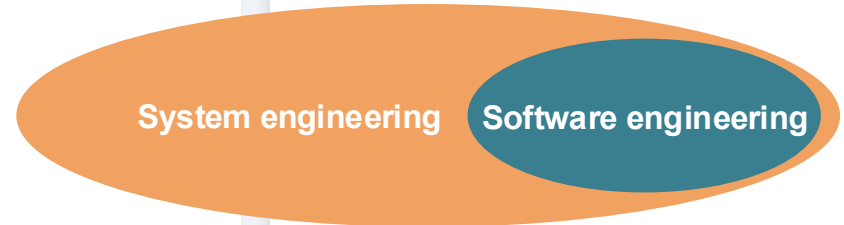
- Focuses on theory and fundamentals

Software Engineering

- Concerned with the practicalities of developing and delivering useful software.
- Sits inside system engineering

System Engineering

- Concerned with all aspects of computer-based systems development including hardware, software and process engineering.
- SE is part of this broader process



Importance of Software Engineering



Society depends on software

More and more, individuals and society rely on advanced software systems. We need to be able to produce reliable and trustworthy systems economically and quickly.



Engineering methods pay off

It is usually cheaper, in the long run, to use software engineering methods and techniques for software systems rather than just write the programs as if it was a personal programming project.



Maintenance dominates costs

For most types of system, the majority of costs are the costs of changing the software after it has gone into use.

3

Software Process

Activities, models, and application diversity

What is a Software Process?

A sequence of activities that leads to the production of a software product

Four fundamental activities common to ALL software processes:



Specification

Customers and engineers define the software to be produced and the constraints on its operation



Development

Designing and programming the software – translating specification into executable code



Validation

Checking that the software does what the customer requires through testing and reviews



Evolution

Modifying the software to reflect changing customer and market requirements after deployment

Key Challenges in Software Engineering



Heterogeneity

Systems must operate across diverse networks, platforms, and devices – from embedded sensors to global cloud infrastructure.



Business & Social Change

Software must adapt rapidly as economies evolve and new technologies emerge, requiring fast, continuous delivery.



Security & Trust

As software is intertwined with all aspects of life, ensuring trustworthy, secure systems becomes critical – failures have real consequences.



Scale

Software has to be developed across a very wide range of scales, from very small embedded systems in portable or wearable devices through to Internet-scale, cloud-based systems that serve a global community.

Software Engineering Diversity

There are many different types of software system and there is no universal set of software techniques that is applicable to all of these.

The software engineering methods and tools used depend on:

- 01 The type of application being developed
- 02 The requirements of the customer
- 03 The background of the development team

Application Types (1 / 2)



Stand-alone

Run locally on a PC or device. No network required. All functionality is self-contained.
Ex: desktop image editors, local CAD tools.



Transaction-based

Run on remote computers; accessed via browsers or terminals.
Ex: e-commerce, online banking, web apps.



Embedded Control

Manage and control hardware devices with strict real-time constraints.
Ex: automotive ECU, pacemakers, washing machines.



Batch Processing

Process large volumes of data in scheduled batches without user interaction.
Ex: payroll systems, bank statement generation.

Application Types (2 / 2)



Entertainment Systems

Intended for personal use or entertainment with high performance & interactivity demands.

Ex: games, streaming, AR/VR.



Simulation & Modeling

Developed to model physical processes or environments, often compute-intensive.

Ex: weather forecasting, physics simulations.



Data Collection

Collect data from the environment via sensors, then relay to processing systems.

Ex: wilderness weather stations, IoT sensors.



Systems of Systems

Composed of multiple independent software systems working together as a unified whole.

Ex: national air traffic management.

Software Engineering Fundamentals

Some principles apply to ALL types of software systems regardless of development approach:

- 01** **Managed Development Processes**
Systems should be developed using a managed and understood development process. Of course, different processes are used for different types of software.
- 02** **Dependability & Performance**
Dependability and performance are important for all types of system.
- 03** **Specification & Requirements**
Understanding and managing the software specification and requirements are important.
- 04** **Software Reuse**
Where appropriate, you should reuse software that has already been developed rather than write new software.

Web-based Software Engineering



The Web is now a platform for running applications and organizations are increasingly developing web-based systems rather than local systems.



Web services allow application functionality to be accessed over the web.



Cloud computing is an approach to the provision of computer services where applications run remotely on the 'cloud'. Users do not buy software but pay according to use.



Web-based systems are complex distributed systems but the fundamental principles of software engineering discussed previously are as applicable to them as they are to any other types of system.



The fundamental ideas of software engineering apply to web-based software in the same way that they apply to other types of software system.

Web Software Engineering – Key Concepts



Software Reuse

Most web systems are built by configuring and integrating existing components and services rather than developing new software from scratch.



Incremental and Agile Development

It is now recognized that it is impractical to specify all the requirements for web-based systems in advance – rapid, iterative releases are the norm.



Service-oriented Systems

Web software is often built as a set of services that communicate via standard protocols, enabling integration across organizational boundaries.



Rich Interfaces

Browser technology such as AJAX and HTML5 now supports the creation of rich interfaces within a web browser, approaching the experience of native desktop apps.

4

Software Engineering Ethics

Responsibilities beyond technical skills

Software Engineering Ethics

Software engineers must behave in an honest and ethically responsible way – not just follow the law, but act according to broader moral principles that promote trust and protect the public.

Four core areas of professional responsibility:



Confidentiality

Respect the privacy of employers and clients even without formal confidentiality agreements.



Competence

Never misrepresent your skill level or knowingly accept work outside your competence.



Intellectual Property

Be aware of laws governing patents, copyright, and software licensing; always respect IP rights.



Computer Misuse

Never use technical skills to misuse others' systems – writing malware, unauthorized access, etc.

ACM/IEEE Code of Ethics

US professional societies produced 8 principles that members sign upon joining. Engineers commit to making software a beneficial and respected profession.

1

Public

Act consistently with the public interest

2

Client & Employer

Act in the best interest of client and employer, consistent with the public interest

3

Product

Ensure products meet the highest professional standards possible

4

Judgment

Maintain integrity and independence in professional judgment

5

Management

Promote an ethical approach to management of software development

6

Profession

Advance the integrity and reputation of the profession

7

Colleagues

Be fair to and supportive of colleagues

8

Self

Commit to lifelong learning and ethical practice

Questions?

Reading: Sommerville, Software Engineering, 10th Ed – Chapter 1

Nguyễn Thị Minh Tuyền · Faculty of IT, HCMUS

CS300 · Elements of Software Engineering · Week 1